1 Introduction:

This is the report for my Honors Program project.
The goal of this project is to study and test the application of machine learning approaches in the context of edge computing. In particular, the focus was on Reinforcement Learning techniques.

The project was approached in three parts:

- first part, the reinforcement learning topic was studied so to better understand its advantages and limitations
- second part was more focused on studying the edge environment and reason on ways to introduce the reinforcement learning in this setting (inspiration for the environment's design was [1]). In this phase all elements that compose both the environment and the chosen RL approach were discussed until the final version was defined.
- last part of the project was a more practical one in which a simulator for the edge environment was developed so to be able to test the chosen RL Algorithm.

In this report, we will analyze all the design choices behind the definition of the environment and of the algorithms. We will talk about the details of the implementation of the simulator and at the end we will see some of the results obtained by performing some experiments.

2 Reinforcement Learning:

Reinforcement Learning (RL) is a branch of Machine Learning in which the focus is on training an agent to make decisions and take actions in a predefined environment. This environment is represented by a state that changes when the agent performs an action. To each action is associated a reward that depends both on the action and on the state.
The agent learn by exploring the environment, observing the outcomes of its actions, and adjusting its behavior based on the rewards received. Over time, the agent learns to map a state to an action such that performing that action will maximize the agent's cumulative reward.

We can see this basic RL framework as a loop. At each time step t, depending on s_t the state of the environment, the agent performs an action a_t. The agent then receives a reward r_t for performing such action, and the new state of the environment s_{t+1}. This iterative process continues with the goal of **maximizing the cumulative final reward**.

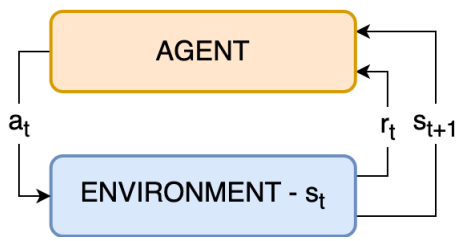

Figure 1: RL Loop

2.1 Mathematical Aspects:

To represent the interaction between the agent and the environment, we can use a mathematical framework such as a Markov Decision Process (MDP); this model allows us to describe a decision-making process in a dynamic system. We can see how the Markov Property holds, indeed, each state of the environment depends only on the previous state and action, not on the entire history of past states and actions. This property simplifies the problem by allowing the agent to make decisions based on the current state alone.

A MDP can be seen as a tuple $< S, A, p, R >$ where:

- S is a finite set of states containing all possible states of the environment
- A is a finite set of action containing all possible actions that the agent can perform
- p is a State Transition Probability Function $p(s'|s, a) = P[S_{t+1} = s'|S_t = s, A_t = a]$ that given as input the current state s and the action a, tells us the probability of transitioning to a new state s'
- R is a Reward function $R(s, a)$ that given a state s and an action a it returns the immediate reward given by performing the action a in the state s

Given this formulation we also have several other important elements to mention:

- **Policy**: the policy π is defined as a function $\pi : S \to A$ that maps a state into an action. This function defines the agent's behavior in a given state. So, given a policy, the new state of the environment depends only on the policy and on the state transition model.
  In general, a policy can be either:
  - *Deterministic*: $a = \pi(s)$
  - *Stochastic*: $\pi(a|s) = P[A_t = a|S_t = s]$

  In the former case given a state s the action chosen following the policy π will always be the same. In the latter, instead, the action depends on a probability distribution, meaning that given a state, the policy could return different actions.

- **Long Term Reward:** is the cumulative sum of rewards obtained by an agent over an extended period of time. It represents the overall performance of the agent and is defined as

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 t_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k} \tag{1}$$

  where $\gamma \in [0, 1]$ is the Discount Factor and determines the relevance of future steps. The long term reward allows the agent to account for the consequences of its action in the future steps and make decisions that lead to a better outcomes over time.

- **State-Value Function:** is the expected future rewards that the agent accumulates starting from a state and following the policy.

$$V_\pi(s) = E[G_t|S_t = s] = E[\sum_{k=0}^{\infty} \gamma^k r_{t+k}|S_t = s] \tag{2}$$

  This function of the states basically tells us how "good" a state s is in the long run.

- **Action-Value Function**: is somehow similar to the state-value function, but in this case we consider the expected reward in the long run starting from a state s, performing action a and the following the policy

$$Q_\pi(s,a) = E[G_t|S_t = s, A_t = a] = E\sum_{k=0}^{\infty} y^k r_{t+k}|S_t = s, A_t = a] \tag{3}$$

  This function is also called *Q-Function* and tells us how "good" is a state s if we perform action a.

- **Bellman Equation**: describes the relationship between the value of a state and the values of its neighboring states. The equation can be written either considering the State-Value Function or the Q Function as follows

$$V(s) = E[R(s,a) + \gamma V(s')] \tag{4}$$
$$Q(s,a) = E[R(s,a) + \gamma Q(s',a')] \tag{5}$$

  where $a = \pi(s), a' = \pi(s')$ are actions carried out according to policy, s is the current state and s' is the state after executing a on s.
  It expresses the value of the state in terms of the immediate reward and the value of the next state.

2.2 Finding the Optimum:

As already mentioned, the agent's goal is to maximize the cumulative reward in the long run. Because the agent follows a policy, the final goal became finding an *optimal policy* π^* such that by following it the agent receives the maximum cumulative reward possible. In other words, it must hold that

$$\pi^* \geq \pi \ iff \ V_{\pi^*}(s) \geq V_\pi(s) \ \forall s \in S \tag{6}$$

Associated to the optimal policy π we have

- *Optimal State-Value Function*: $V^*(s) = \max_\pi V_\pi(s) \ \forall s \in S$
- *Optimal Q-Function*: $Q^*(s,a) = \max_\pi Q_\pi(s,a) \ \forall s \in S, \ \forall a \in A$

Moreover, if we consider the Optimal Q-Function Q^* it must satisfy the *Bellman Optimality Equation* as follows:

$$Q^*(s,a) = E[R(s,a) + \gamma \max_{a'} Q^*(s',a')] \tag{7}$$

this equation tells us that starting from a state s and selecting action a we will obtain the optimal value if we consider as next state-action pair the one that gives us the maximum reward.
From this we can derive that the *Optimal Policy* π^* will be

$$\pi^*(s) = argmax_{a \in A} Q^*(s,a) \tag{8}$$

Our final goal is to learn this Optimal Policy, but in most real use cases both the reward function and the state transition function are unknown. For this reason, we need to learn them in order to be able to estimate the Optimal Q-Function and then the optimal policy.

2.3 RL Algorithms and Approaches:

The first concepts to mention when talking about RL algorithms are *Exploitation* and *Exploration*. Indeed, in training an agent, we should consider the trade-off between the two.
In the former, the agent only uses the already known information to maximize its reward; this means that even if there could be a "path" that could give him more reward, the agent will never find it getting stuck with a smaller reward than the one that is theoretically possible.
In the latter, the agent explores the environment to find new information that allows him to maybe obtain a higher reward, but nothing guarantees that a "path" for a higher reward exists, meaning that the agent may gain nothing from its exploration.
In general an ϵ-greedy strategy is used, which consists in performing a random action with probability ϵ to explore and perform the best action with probability $1 - \epsilon$ to exploit the knowledge we already have. This can be written as:

$$\pi^*(s) = \begin{cases} argmax_{a \in A} Q^*(s,a) & probability\ 1-\epsilon \\ random\ a \in A & probability \epsilon \end{cases} \tag{9}$$

More in general, there are several possible approaches to solve a RL task. These methods differ in what the agent learns. For example, the agent can try to approximate the optimal value function/optimal Q-function (*Value-based*) or the policy (*Policy-based*) or even the environment itself (*Model-based*).
In all of these approaches we could decide to be *On-Policy* or *Off-Policy*, meaning that the agent either strictly follows the policy or may deviate from it.

We will now see some of the most common algorithms to solve RL tasks:

Value-Based Methods: these algorithms estimate the value of different states or state-action pairs and learn a value function. The goal is to find an optimal policy by selecting actions that maximize the expected cumulative reward. Two common ones are:

- State-Action-Reward-State-Action (SARSA): is a model-free, value-based, on-policy approach in which we estimates the action-value function with

  $$Q(s,a) \leftarrow Q(s,a) + \alpha[R(s,a) + \gamma Q(s',a') - Q(s,a)] \tag{10}$$

  Being an on-policy approach, this algorithm tends to prefer exploitation rather than exploration. For this reason, even if it eventually converges, it may take longer than other approaches.

- Q-Learning: is a model-free, value-based, off-policy approach that estimates the action-value function. It differs from SARSA because we are directly approximating Q* independently of the agent actions. (off-policy). The update rule is as follows:

  $$Q(s,) \leftarrow Q(s,a) + \alpha[R(s,a) + \gamma \max_a Q(s',a') - Q(s,a)] \tag{11}$$

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Choose A from S using policy derived from Q (e.g., ε-greedy)
  Loop for each step of episode:
    Take action A, observe R, S'
    Choose A' from S' using policy derived from Q (e.g., ε-greedy)
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$
    $S \leftarrow S'$; $A \leftarrow A'$;
  until S is terminal

Figure 2: SARSA Learning Algorithm from [2]

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Take action A, observe R, S'
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma \max_a Q(S', a) - Q(S, A)\big]$
    $S \leftarrow S'$
  until S is terminal

Figure 3: Q-Learning Algorithm from [2]

Both these algorithm are usually implemented using a Tabular approach, meaning we maintain in memory a table, called *Q-Table*, that stores all action-value estimates. For this reason, standard Q-Learning may struggle with a large set of states because the Q-Table would become so large that it would be unmanageable.
So, instead of using a tabular approach, we introduce a Deep Neural Network whose role is to approximate the Q-function. The Neural Network takes as input the state and outputs the Q value for each action, then the agent selects the one with higher Q value. We call this a *Deep Q-Network* (DQN). Starting from this simple initial concept several other more complex architectures have been developed, for example, the *Double DQN*.

Policy Gradient Methods: are a type of algorithm in which we do not learn the Q-function but a *parametrized policy function* π_θ. This policy function is then optimized in such a way that, given a state, it directly returns the optimal action. We can also write this parameterized policy as

$$\pi(a|s;\theta) = P[A_t = a|S_t = s, \theta_t = \theta] \tag{12}$$

expressing it as the probability that action a is taken at time t given that the environment is in state s with parameter θ.
With this type of algorithm we follow a more 'standard' ML approach, in fact, we consider an

objective function that we want to maximize and some parameters that we update using the gradient of the objective function. Here an example of common objective function and parameterized policy:

$$J(\theta) = V_{\pi_\theta}(s_0) \qquad \pi_\theta(s,a) = \frac{e^{\Phi(s,a)^T\theta}}{\sum_{k=1}^{N} e^{\Phi(s,a_k)^T\theta}}$$

Hybrid Approaches: These are basic techniques, but the most recent algorithms opt for a hybrid approach that uses both policy gradient and values-based methods.

A family of these algorithms is called **Actor-Critic**. These approaches simultaneously attempt to learn a policy (the actor) and estimate the value function (the critic) to improve the decision-making process of an agent in an environment.
The interaction between the actor and the critic allows iterative improvement of the policy. The actor uses the critic's value function estimates to guide its action selection, while the critic uses the actor's policy to evaluate different state-action pairs and update its value function estimates. This interaction enables the actor to explore and exploit the environment more effectively.
In the image below, we can see the base actor-critic algorithm

One-step Actor–Critic (episodic), for estimating $\pi_{\boldsymbol{\theta}} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$
Input: a differentiable state-value function parameterization $\hat{v}(s,\mathbf{w})$
Parameters: step sizes $\alpha^{\boldsymbol{\theta}} > 0$, $\alpha^{\mathbf{w}} > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)
Loop forever (for each episode):
  Initialize S (first state of episode)
  $I \leftarrow 1$
  Loop while S is not terminal (for each time step):
    $A \sim \pi(\cdot|S,\boldsymbol{\theta})$
    Take action A, observe S', R
    $\delta \leftarrow R + \gamma\hat{v}(S',\mathbf{w}) - \hat{v}(S,\mathbf{w})$ (if S' is terminal, then $\hat{v}(S',\mathbf{w}) \doteq 0$)
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}}\delta\nabla\hat{v}(S,\mathbf{w})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} I\delta\nabla\ln\pi(A|S,\boldsymbol{\theta})$
    $I \leftarrow \gamma I$
    $S \leftarrow S'$

Figure 4: Actor-Critic Algorithm from [2]

Starting from this base implementation several variations were introduced between which some of the more relevant are: Advantage Actor-Critic (A2C) and Proximal Policy Optimization (PPO). The latter will be used in the practice part; for this reason, it will be explained in more depth.

A2C: using Actor-Critic as its base introduces the advantage function, which measures the advantage of taking a specific action in a given state; it reduces the variance of the actor's policy updates. This reduction in variance leads to more stable learning and helps the agent explore and exploit the environment more effectively.

PPO: aims to update the actor's policy in a more reliable and efficient manner by utilizing a surrogate objective function. It constrains policy updates to a trust region, ensuring that the updated policy does not deviate too far from the previous policy. This constraint prevents large policy changes that could result in instability during training.
PPO introduces a clipping mechanism, known as "clip loss," which bounds the policy update to a range defined by a hyperparameter. This range prevents the policy from changing too drastically and provides a more conservative and stable update. By constraining policy changes, PPO can improve the sample efficiency and achieve more reliable learning compared to other policy gradient methods.
Additionally, PPO incorporates multiple epochs of mini-batch updates, allowing for more efficient utilization of the collected experiences. It uses multiple passes over the collected data to perform several iterations of policy updates, enhancing the learning process. This approach helps stabilize training and ensures that policy optimization is performed on more diverse and informative samples. It should be mentioned that PPO trains a stochastic policy, meaning that exploration is performed simply by sampling actions following its stochastic policy

Algorithm 1 PPO-Clip

1: Input: initial policy parameters θ_0, initial value function parameters ϕ_0
2: **for** $k = 0, 1, 2, \ldots$ **do**
3: Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.
4: Compute rewards-to-go $\hat{R}_t$.
5: Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{ϕ_k}.
6: Update the policy by maximizing the PPO-Clip objective:

$$\theta_{k+1} = \arg\max_{\theta} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \min \left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t), \quad g(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)) \right),$$

typically via stochastic gradient ascent with Adam.
7: Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg\min_{\phi} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \left(V_\phi(s_t) - \hat{R}_t \right)^2,$$

typically via some gradient descent algorithm.
8: **end for**

Figure 5: Version of PPO Algorithm from [3]

3 Practice Part

To apply these techniques, we chose the case of communication between edge nodes. Such an environment gives us several challenges that could be tackled with reinforcement learning. We will now explore in more detail the environment and its elements, the technologies used, and some of the design choices.
Because we wanted to focus on the use of RL techniques, we decided to implement the environment in a simulator instead of using real devices. This allowed for greater flexibility, avoiding issues that may arise when working with real hardware such lack of devices to use, software incompatibilities between slightly different hw, connections issues. Clearly, these are problems that in a real setting would have to be addressed, but for our purposes can be ignored.

Another important decision that affected development was the choice to implement an **online** reinforcement learning approach. This means that at the start of the environment lifetime the agent knows nothing and it learns by playing directly in the environment, this is in total contrast to an *offline* learning approach were the agent learn from a fixed dataset of sample and then with the acquired knowledge starts playing.
With an online approach we clearly need more time before the agent learns enough to give us an advantage over the offline approach but it allows for more flexibility and adaptability in a real world scenario, where the elements that affect that state may change not in a constant way, for example the rate at which tasks arrive at a node may change with time.

3.1 Environment:

Our environment is composed of a set of N nodes, a *Task generator* and a *Retransmission Service*. Each node is able to communicate with all the other nodes and its goal is to execute the tasks that are assigned to it **within a deadline**.
A node i can receive a task in two possible ways:

- the task generator creates a task with destination node i
- the retransmission service relays to i a task that another node j offloaded to i.

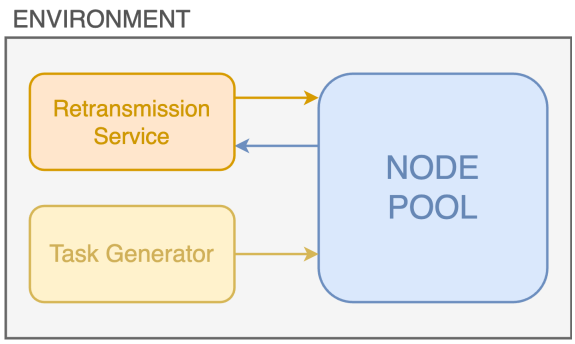

Figure 6: the environment

3.2 Node:

As mentioned above, the goal of a node is to execute tasks but we also need to talk about the constrains and limitations of the node in trying to achieve its goal and consequently the aspect we try to solve using a RL agent. Reasoning about the main challenges of edge computing, we concluded that the main constrains are:

- capacity of the battery of the node
- time in which the task must be completed

So, at first we reasoned about trying to maximize the life span of the device while at the same time trying to execute "in deadline" as much task as possible. This seemed a good approach, but we soon realized that it would mean trying to optimize two aspects at the same time, making the problem much more complex.
For this reason, we decided that the agent's goal is to learn how to **maximize the number of task that are processed within their deadline**.

We then analysed the possible actions that a node could take and came out with:

- **Execute Locally:** the task is inserted in the node's internal queue and will be executed by the note itself;
- **Offload:** the node decide that it cannot execute the task and sends it to another node in the environment

Note how in this way a node can not offload a task to itself. It should be mentioned that during the study of possible actions, this was not always the case. Indeed, we also consider giving the ability to a node to offload a task to itself. The idea was that in this way the node would be able to postpone the decision over a task; maybe at that precise moment the node could not execute it but the task's deadline is far away in time and the node may want to reevaluate the task at a later time.
This approach was discarded because it led unwanted behaviour such as: the node keeps offloading a task to itself entering in a loop from which it does not gain anything.

Another important point of discussion was how to represent the state of a node. So, taking into account the node's goal of completing as many tasks as possible within the deadline. We decided that the **state** would be represented by not only the internal state of the node but also by characteristics of the analyzed tasks. After these considerations, we defined that state as:

- the remaining time before the deadline of the task,
- the time needed to execute the task,
- the elements that are in queue waiting for the *final* processing [1]

In this way, the model have all the relevant information about the node and the currently task. It should be noted that in this way, the node has a new state every time a new task is handled.

[1] these are the task that the agent already decided to execute locally

Taking instead about the structure of a node, it has five main components: Model, Message Handler, Service Queue, Processing Queue and Processor.

Service Queue: a FIFO queue where all incoming tasks are stored.
Model: takes decision on a task taking into account the state of the node and the task itself.
Message Handler: component that handles incoming tasks, is the component that interacts with the model to choose how to handle a task.
Processing Queue: a Priority Queue where tasks that will be executed locally are stored. As mentioned, this is a priority queue where tasks are ordered according to the time they have left before deadline.

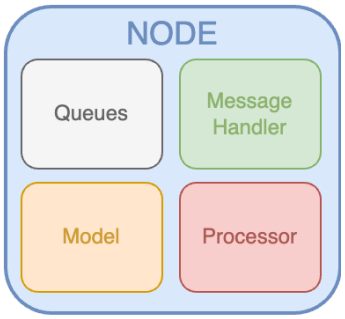

Figure 7: node structure

At this point, we can see how a task is processed by a node.
As soon as a task arrives at a node, it is inserted into *Service Queue*. Then using a FIFO logic the tasks are extracted by the *Message Handler*.
Once the *Message Handler* has a new task, it updates the state of the node (NOTE: the new state is also stored in a log file) then the handler passes the new state to the model that returns the action that needs to be performed. If the chosen action is to offload, then the task is sent to the *Retransmission Service* that will retransmit the task to its destination. Otherwise, if the task is to be executed locally, it is inserted in the *Processing Queue*. Finally, once the task is the one with the smallest remaining time and the *Processor* is not working, the task is extracted from the queue and is processed.

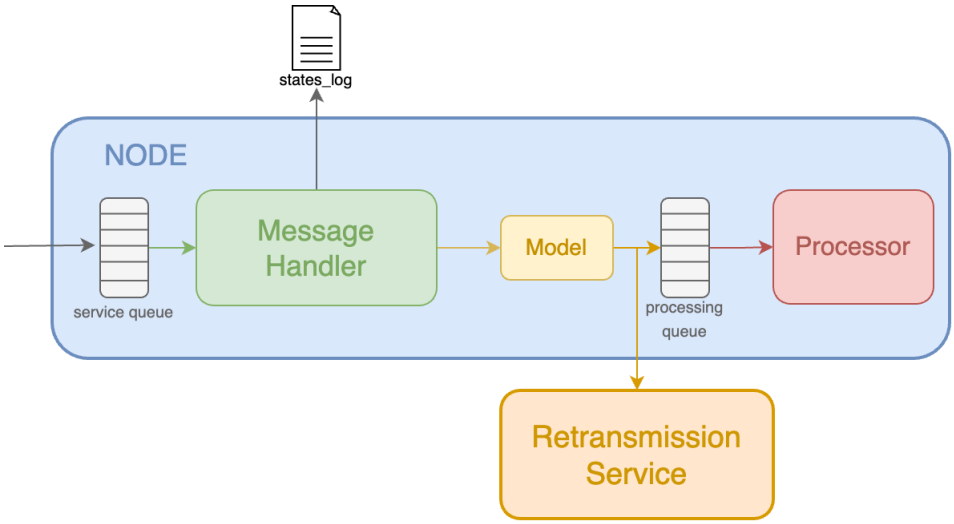

Figure 8: task lifespan

When talking about processing a task is possible to better analyze the online approach mentioned before. After a certain number of processed task, the model is updated. What happens can be seen as a sort of fine tuning of the model, indeed, the weights of the model are updated using as sample for training the last batch of tasks.
Last thing to mention about a node is that it does not cooperate with other nodes in the environment. This means that each node tries to maximize the number of task within deadline independently from all the other nodes.
Using a non cooperative setting, would in theory bring nodes towards a behaviour in which they try to penalize other nodes. Clearly, this approach would be counterproductive for the environment. To mitigate the risk of nodes preferring to give a disadvantage to an adversary than executing task, the reward function was carefully constructed.

3.3 Reward Function:

We can represent the state s as the tuple (r, e, q) where we have:

- r the remaining time before the task's deadline
- e the expected time needed to complete the task
- q the number of elements in the *Processing Queue*

then considering also ret the time needed to transmit a task between nodes, and depending on the chosen action a. It is possible to write the reward as:

$$R((r, e, q), a) = \begin{cases} r - e - q & a = execute\ locally \\ r - e + q - ret & a = offload \end{cases} \tag{13}$$

We can now analyze it. The basic idea behind the reward function is to give a (positive) reward when the chosen action allows the node that will process the task (either the one that made the decision or the one that received the task after the offloading) to execute the task within the deadline. In more detail, this reward is directly proportional to how much in advance the task is executed.
On the other hand, the agent receives a penalty when it decides to perform an action that causes a task to end its processing after the deadline; again, this penalty is directly proportional to how late the task is completed.
Starting from this idea, we decided that more factors needed to be considered.
First, when the node offloads a task, it takes some time for the task to arrive at the new node, which means that it will have less time to complete the task. For this reason, we added the element ret, and indeed, by subtracting the retransmission time, we are making the agent aware of the disadvantages of performing the offlaod action.
We can see how this simple approach encourages the agent to prefer the "execute locally" action. This can be an advantage because it reduces the risk of tasks being sent back and forth between nodes. However, this approach will inevitably lead the agent towards always choosing to execute locally. We can clearly see this because $r - e > r - e - ret$.
In the long run, this clearly leads to a pile-up of task in the *Processing Queue* that will likely bring many tasks over the deadline.
The solution found to mitigate this issue is to add a term q that depends on the number of elements

present in the *Processing Queue*. In particular, by subtracting this term when executing locally and by adding when offloading, we are encouraging the agent to choose local execution only if there are few elements in the processing queue and to choose offloading when the queue has many elements.

This reward function should be a good compromise of what we want our agent to learn. In theory, using this reward function, the agent learns to balance between executing tasks locally and offloading them to another node depending on the current load of the node with the final goal of maximizing the number of tasks whose processing is terminated within the deadline.

3.4 Implementation Details:

As mentioned in the introduction of Chapter 3, to test the environment and the reinforcement learning approaches, a simulator was built. In detail, the simulator was built using the *SimPy* python library, meanwhile the RL algorithms were implemented using the *Gymnasium* and *Stable Baseline 3* python libraries.

Simpy: is a versatile simulation library that offers a framework to model and simulate complex systems efficiently. SimPy excels at discrete event simulation and provides a wide array of tools and features. The main components that we used to build the simulator are:

- *Processes*: are simple python generators whose role is to create events and "yield" them in order to wait for them to be triggered. This event could be either a timeout or a new element is added to a queue.
  In this simulator *processes* are used to implement the following environment components: Task Generator and Retransmission Service. And these functionalities of a node: message handling and processing. Indeed, all of these elements can be seen as infinite loops where the process waits for something, for example, the task generator waits a timeout before generating a new task, while the message handler waits for a new element added in the *Service Queue*.

- *Stores*: objects in which it is possible to store and retrieve objects. In our case, they are used to implement all the queues. To be more precise for all FIFO queue (*Service Queue* and the queues that handle retransmissions), the base store implementation was used. Meanwhile, for the *Processing Queue* that is a priority queue, a PriorityStore was used, that allows to implement the priority aspect simply by wrapping the task in a PriorityItem object.

Gymnasium: a standard API for reinforcement learning and a diverse collection of reference environments. It provides tools and functionalities to build and interact with custom RL environments. We used it to define observation and action spaces, using simple spaces or a composition of simple spaces. The most common simple spaces are:

- *Discrete*: describes a discrete space where {0, 1, . . . , n-1} are the possible values our observation or action can take,

- *Box*: describes an n-dimensional continuous space. It is a bounded space where we can define the upper and lower limits of the values the observation can take.

In detail, our observation space is a *Dict* object that contains the following elemental spaces: a *Discrete* space to represent the number of elements in the queue, and two *Box* spaces to represent

the remaining time and the expected time. Instead, the action space is represented by a single *Discrete* space.
Stable Baseline 3: provides a set of implementations of reinforcement learning algorithms in PyTorch. It's algorithms can be used in combination with a custom Gymnasium environment. We use it to easily implement and handle the PPO algorithm that is used to make decisions.

In conclusion, we should talk about an aspect whose importance was discovered only during the experiments. We noticed that after some iterations in the environment, the number of elements in queue would start becoming very high, moreover, the rewards assigned to the node would start becoming very low. After some debugging, we discovered that some extremely late task would pile up in the queues slowing down the analysis of other tasks and and consequently making them late. To solve this issue, we gave to the node the ability to **discard** terminally late tasks. Indeed, it does not make sense to ask the model how to process the task because the only thing that could happen is to make the task even more late For this reason, before asking the model, the node checks if the task is not too late, and if it is, the task is discarded.
This allows the node to focus on tasks that have the possibility of being executed within the deadline, giving in the long run a benefit on the performance of the agent.

3.5 Experiments:

All experiments were performed using the free GPU provided by Google Colab. Due to the limited computational resources and to simplify the study of the environment, we decide to fix some global parameters. The environment has a life of 50000 ticks after which the simulation stops. There are 3 identical nodes in the node pool of the environment each has a unique ID and updates its model every 20 tasks. Task are generated at a different rate for each node: node_0 receives a task every 3 ticks, node_1 every 5 ticks and node_2 every 7 ticks.
Each task need 5 ticks to be processed and has a deadline 10 ticks after it was generated and if the discard ability of the node is active a task is discarded after 20 ticks after its deadline.
It should be noted that because the rate of arrival of tasks to nodes is different each node will process a different number of tasks.

Using these parameters, four experiment were performed, in which we analyzed how model and the discard ability influences the results. So we have:

1. model and discard ability TURNED OFF
2. model and discard ability TURNED ON
3. model TURNED OFF and discard ability TURNED ON
4. model TURNED ON and discard ability TURNED OFF

We then evaluate the results not using the reward but studying the number of tasks that are processed within their deadline. This mainly for an easier understanding of the agent's trend, indeed in our use case the fact that the reward has a certain value does not precisely tells us how good the agent is in achieving our final goal of maximizing the number of tasks executed within deadline.
Moreover, another aspect that we analyze are the policy of each node and how they evolve during

the environment's lifetime.
In conclusion, we should mention that in the implementation all the actions have the name "offload_ID" where ID is the identifier of a node. When the ID is equal to the identifier of the node that made the decision, then the node is offloading to itself meaning it decided to "execute locally" the task.
No Model - No Discard: In this case due to the fact that there is no model, the policies will be random and each of the 3 possible actions will be chosen with probability of $\frac{1}{3}$ so effectively there is no policy.

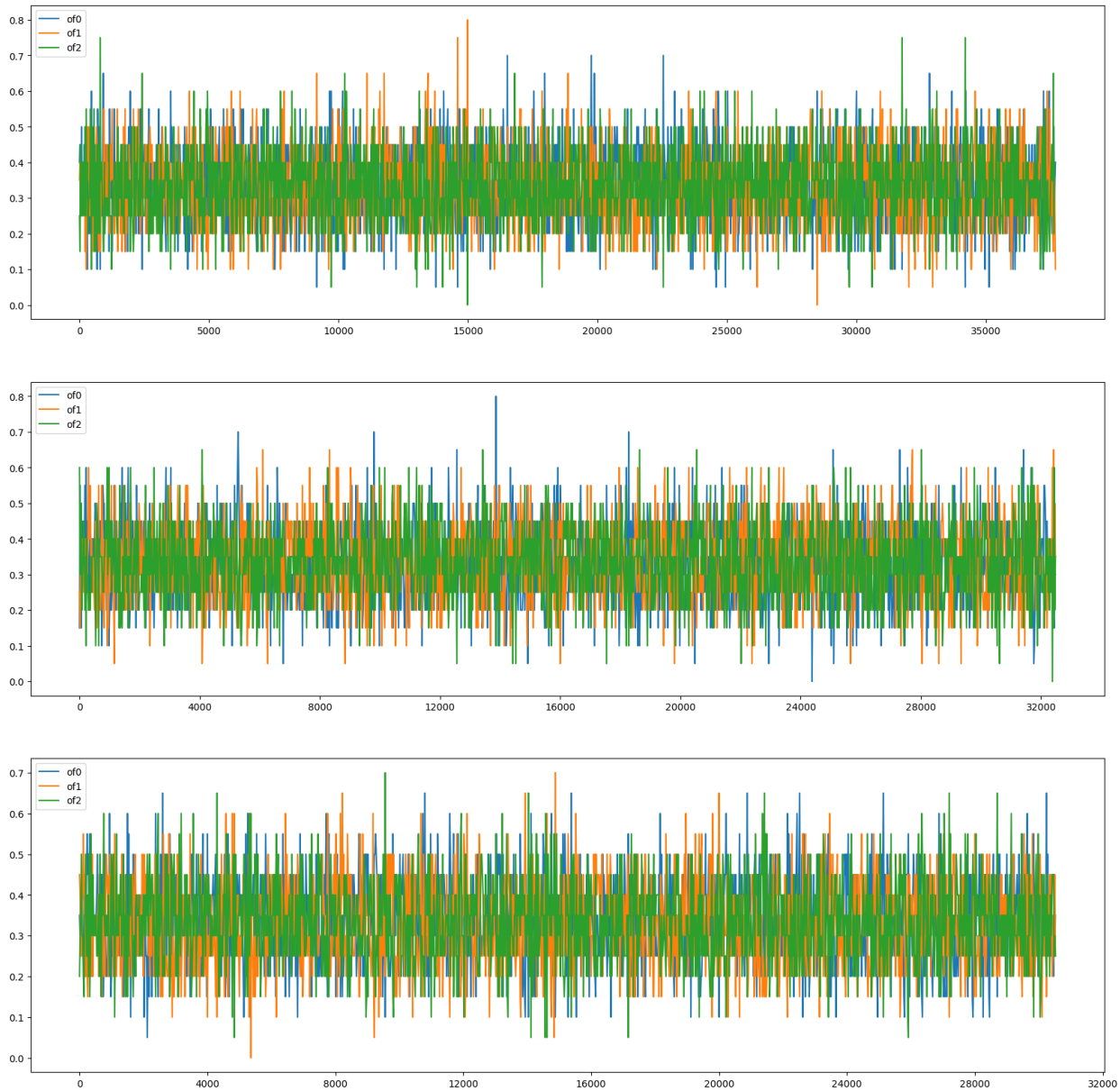

Figure 9: Policy Plots for No Model No Discard Case

We can see this randomness also by counting the how many times a node decided to perform an action when processing a task. As we can see in Table 1. in each node the action were chosen more or less the same number of time and because there are only three actions this clearly follows a random policy where each action has probability of $\frac{1}{3}$

	Offload_1	Offload_2	Offload_3
Node 1	12640	12531	12527
Node 2	10707	10925	10868
Node 3	10325	9970	10242

Table 1: This table shows for each node how many times a certain action has been chosen for processing a task. Note that the highlighted elements in the main diagonal represent the node offloading to itself so it represents when the node decide to execute the action locally

Clearly this random approach does not give us any benefit, indeed practically all of the task went out of deadline as shown in Table 2. This is probably due to a combination of the absence of the model and that tasks pile up in the queue because the discard ability is turned off.

	In Time	Late	Discarded
Node 1	7	9992	0
Node 2	16	9980	0
Node 3	59	9930	0

Table 2: This table shows for each node how many tasks were completed in time, how many were late and how many were discarded

No Model - Discard: In this experiment, we tested if the discard ability introduce any improvement. Clearly because there is no model, the policies of each node are random and each action is chosen with a probability of $\frac{1}{3}$ as is possible to deduce from Table 3.

	Offload_1	Offload_2	Offload_3
Node 1	12464	12642	12621
Node 2	10790	11130	10816
Node 3	10275	10097	10205

Table 3: Number of times an action has been chosen in the case of no-model and discard

But in this case there is a slight improvement in the number of tasks that are executed in time, probably thanks to the fact that some tasks are discarded and do not pile up in the queues. We can see this in Table 4.
Only for this experiment, we will not show the plots of the policies because they are identical to ones shown in the "No-Model No-Discard" experiment.

	In Time	Late	Discarded
Node 1	454	9510	2503
Node 2	1413	8400	1308
Node 3	6002	1861	249

Table 4: This table shows for each node how many tasks were completed in time, how many were late and how many were discarded

Model - No Discard: In this experiment, we introduce for the fist time and try to understand if it is enough to cope with the task pile up problem. With a first look to the policies in the image below and from Table 7, we could conclude that the model is enough and that the model is learning an optimal policy.

	Offload_1	Offload_2	Offload_3
Node 1	3540	34881	11518
Node 2	46452	2623	805
Node 3	2005	10779	6681

Table 5: Number of times an action has been chosen in the case of model and discard

But if we look at the data about the number of tasks executed within the deadline, we can see that the pile up of tasks is still a big issue that make many other tasks late.

	In Time	Late	Discarded
Node 1	25	3515	0
Node 2	10	2614	0
Node 3	6219	462	0

Table 6: This table shows for each node how many tasks were completed in time, how many were late and how many were discarded

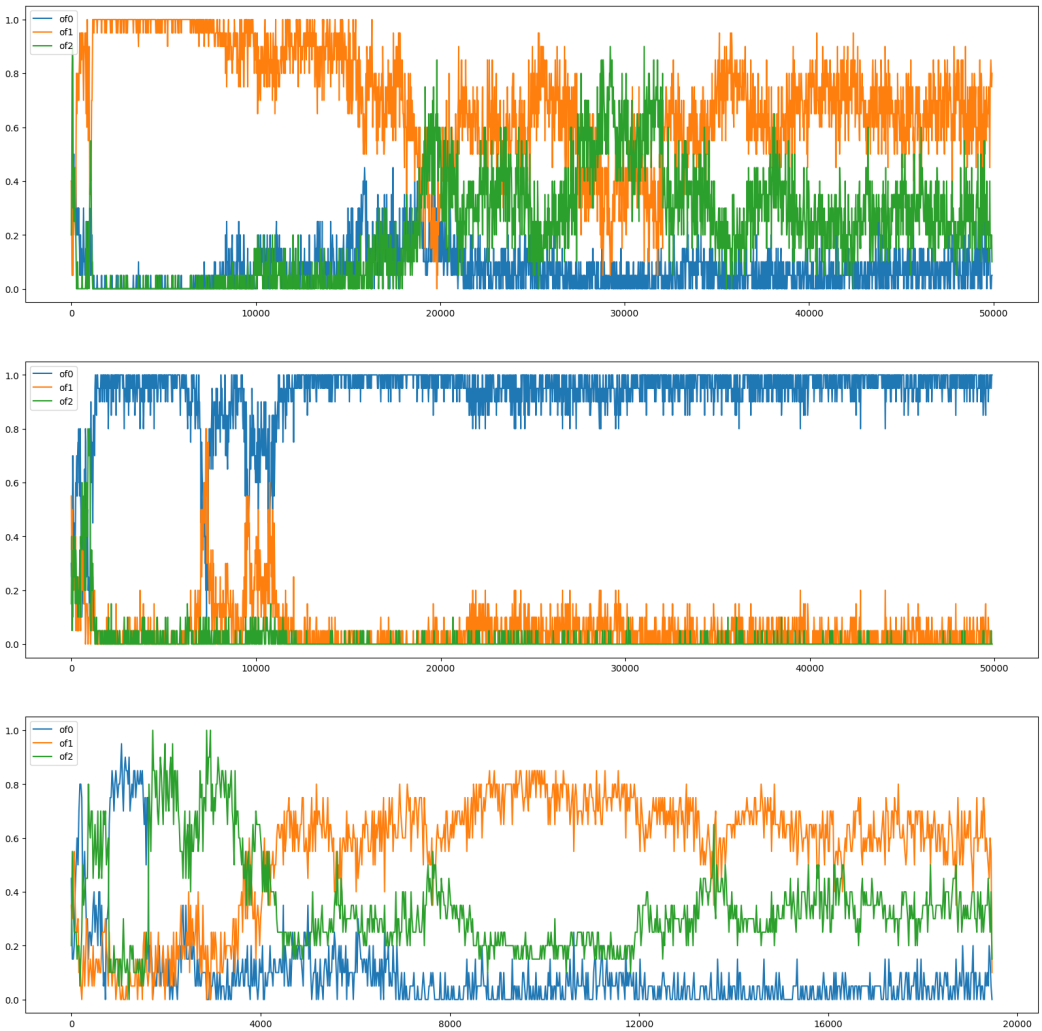

Figure 10: Policies for the Model - No Discard case

Model - Discard: In this final experiment, we test how the model works in combination with the discard ability. We fist analyze the policies, before starting is important to note that even if the plots have the same dimensions, they have **different values on the x axis**. This is due to the fact that each node receives tasks at a different rate.
Starting from the top, we have the node that receives the tasks more frequently, we can see that it tends to prefer offloading tasks to other nodes (yellow and green lines) even if it also tries to execute some of them locally (blue line). On the second plot instead, we have the node with an intermediate arrival rate (Note how the arrival rate is equal to the time needed to process a task), so it needs a little bit to learn that executing locally (yellow line in this case) is the best action to perform. Lastly, the final plot is for the node with the lower rate of arrival of task, in this case the rate is grater than the time to execute a task. This means that the agent quickly learn that the best action is to execute locally.

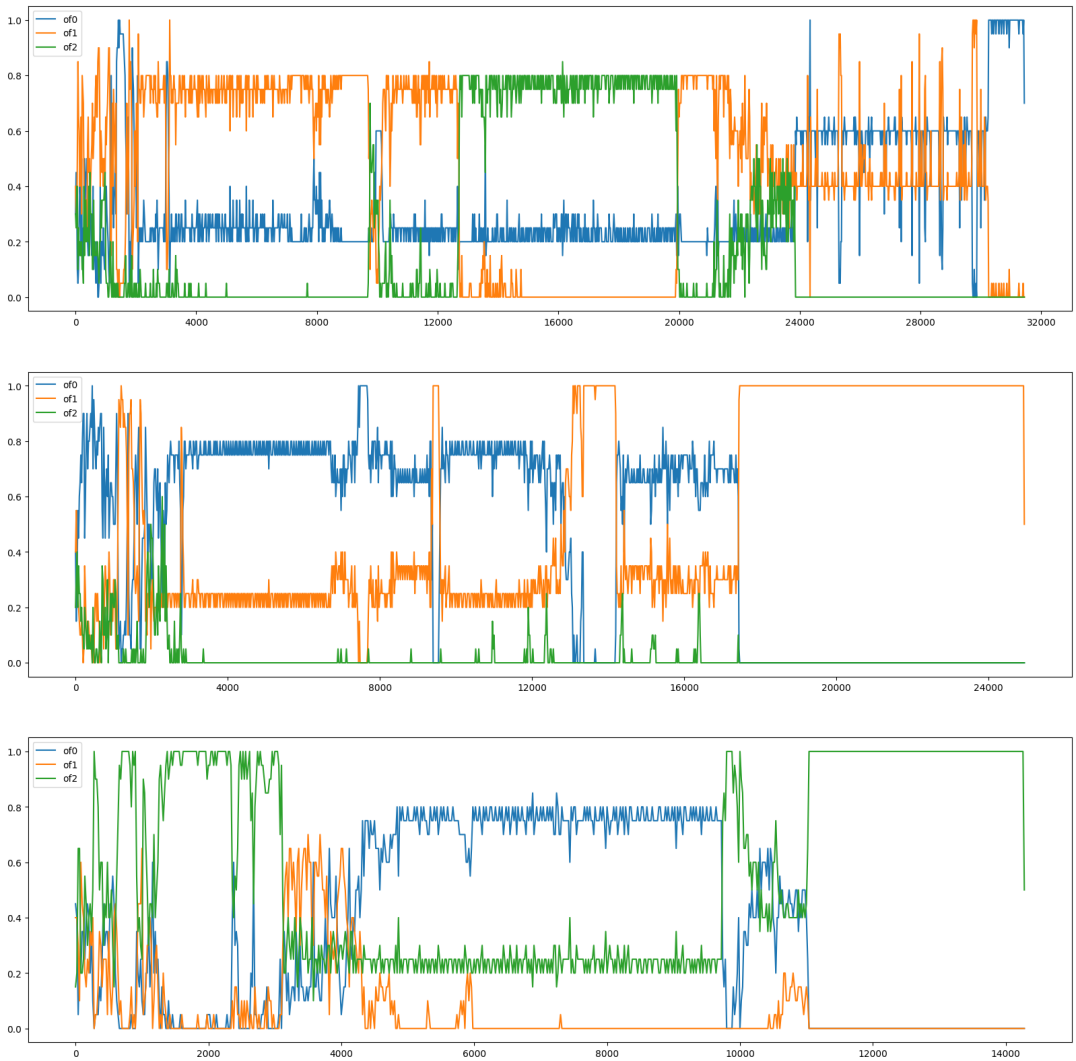

Figure 11: Node's policies for the case model - discard

Studying the other collected metrics in Table 5 and 6, we can see that there is an improvement by using the model in combination with the discard ability. Indeed, the number of task that are completed within their deadline is increasing.
We could assume that given more simulation time these results could improve even more.

	Offload_1	Offload_2	Offload_3
Node 1	10667	14054	6733
Node 2	11205	13308	437
Node 3	5206	994	8090

Table 7: Number of times an action has been chosen in the case of model and discard

	In Time	Late	Discarded
Node 1	4341	5195	744
Node 2	3098	6652	3650
Node 3	6002	1861	249

Table 8: This table shows for each node how many tasks were completed in time, how many were late and how many were discarded

4 Conclusion and Future works

This work allowed us to better understand how to approach the introduction of reinforcement learning approaches in a edge computing setting. It also allowed us to see that a improvement over the standard approach is possible even if it has some limitations that were briefly discussed in this report.
Moreover, the practice part allowed us to study how the theoretical part could be transferred in a real environment, by also putting the spotlight on problems that were not evident during the theoretical discussion. In conclusion, this work can be the starting point for further development and studies on what is the influence of the model update frequency, or how the model behaves within a more dynamic environment where execution times and availability of nodes may change.

References

[1] Gabriele Proietti Mattia and Roberto Beraldi. "On real-time scheduling in Fog computing: A Reinforcement Learning algorithm with application to smart cities". In: *2022 IEEE International Conference on Pervasive Computing and Communications Workshops and other Affiliated Events (PerCom Workshops)*. 2022, pp. 187–193. DOI: 10.1109/PerComWorkshops53856.2022.9767498.

[2] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd. Cambridge, MA: MIT Press, 2018.

[3] OpenAI. *Proximal Policy Optimization Algorithms*. URL: https://spinningup.openai.com/en/latest/algorithms/ppo.html#pseudocode (visited on 06/07/2023).

[4] Haochen Hua et al. "Edge Computing with Artificial Intelligence: A Machine Learning Perspective". In: *ACM Comput. Surv.* 55.9 (Jan. 2023). ISSN: 0360-0300. DOI: 10.1145/3555802. URL: https://doi.org/10.1145/3555802.